

RAG на стероидах

от семантического поиска до автономного агента

Кирилл Сухорук

Senior Fullstack Developer | SoftLight

whoami

Кирилл Сухоруких

Senior Fullstack Developer @ SoftLight

- .NET / TypeScript / Python
- Вайб-кодинг энтузиаст

Сегодня покажу

- Реальную продакшн RAG-систему
- Гибридный поиск: SQL + Embeddings
- MCP-протокол для LLM
- Мульти-агентную оркестрацию
- Feedback loop: самообучение
- Цифровой двойник с клоном голоса
- Готовый код — не tutorial!

Кому будет полезна эта презентация?

Backend-разработчики

.NET / Python / Go

Строят внутренние инструменты и хотят добавить AI-поиск в существующий стек.

RAG, гибридный поиск, агенты — всё с реальным кодом.

Tech leads и Архитекторы

Оценивают AI-инструменты для команды

Принимают технические решения: self-hosted vs cloud, какой стек, как измерить ROI, что реально работает в продакшне.

AI / ML энтузиасты

Хотят понять RAG и агентов изнутри

Не tutorиалы с игрушечными примерами — а продакшн-система с 50K коммитами, MCP, feedback loop.

DevOps / Platform Engineers

Деплой AI-стека

Qdrant + PostgreSQL + Ollama в Docker, self-hosted без vendor lock-in, на своём железе.

Проблема: разбросанные знания

Jira

3 000+ задач

Бэклог, спринты,
история решений

Confluence

500+ страниц

Документация

Git

**50 000+
КОММИТОВ**

Код, PR reviews,
история изменений

Slack

∞ сообщений

Обсуждения, решения,
логика «почему так»

Инженеры тратят ~30% рабочего времени на поиск информации,
которая уже существует, но разбросана по разным системам

Что если AI-ассистент...

1

...получит доступ ко всей базе знаний команды?
Задачам, документации, коду, истории решений и обсуждениям

2

...не просто найдёт контекст, а начнёт действовать?
Создаст тикет, уведомит инженера, предложит fix из прошлого опыта

3

...станет цифровым двойником инженера?
Не заменой — усилением. Коллега, который помнит каждый коммит за 3 года

План доклада



Часть 1

~15 мин

Классический RAG: теория и ограничения



Часть 2

~15 мин

Гибридный поиск: PostgreSQL FTS + Qdrant Vectors



Часть 3

~10 мин

МСП-протокол: знания как инструменты LLM



Часть 4

~10 мин

Автономный агент: от поиска к действию



Бонус

~10 мин

Цифровой двойник с голосом и личностью

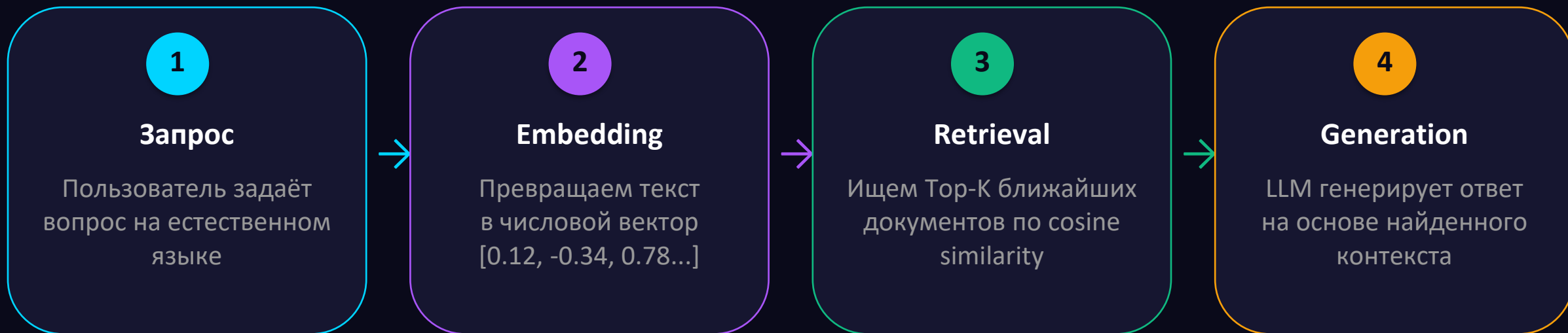
Часть 1

Классический RAG

теория, embeddings и ограничения

Что такое RAG?

Retrieval-Augmented Generation — паттерн, при котором LLM получает релевантный контекст из внешней базы знаний перед генерацией ответа (Lewis et al., 2020, Meta AI)



Зачем RAG, если LLM и так умные?

- ▶ LLM не знает внутренних процессов вашей компании, вашего кода, ваших решений
- ▶ Контекстное окно ограничено — нельзя загрузить 50K коммитов в промпт
- ▶ RAG = «подсказка шпаргалкой» — LLM получает только релевантный кусок, а не всю библиотеку

Теория: Embeddings — превращаем текст в числа

Embedding — это числовой вектор фиксированной размерности, который кодирует семантический смысл текста.

Похожие по смыслу тексты → близкие векторы.

"Как оплатить заказ?" →

[0.12, -0.34, 0.78, 0.02, ...] (768 чисел)

"How to process payment?" →

[0.11, -0.32, 0.76, 0.04, ...] (768 чисел)

↑ Близкие векторы! Cosine similarity = 0.94

"Рецепт борща" →

[-0.45, 0.67, -0.12, 0.89, ...]

↑ Далёкий вектор. Cosine similarity = 0.08

Модели embeddings + аналогия для .NET

Какую модель выбрать?

Модель	Размерность	Где работает	Цена
nomic-embed-text	768D	Ollama (локально)	Бесплатно
text-embedding-3-small	1536D	OpenAI API	\$0.02 / 1M tokens
text-embedding-3-large	3072D	OpenAI API	\$0.13 / 1M tokens

- Похожие строки дают похожие массивы
- Работает кросс-язычно (RU $\approx$ EN)

Теория: Векторные базы данных

Что хранит Vector DB?

```
ID:      uuid-v4
Vector:   [0.12, -0.34, 0.78, ...] × 768
Payload:  {
  source_type: "jira",
  source_id:   "PROJ-123",
  content:     "Implement payment...",
  content_hash: "sha256..."
}
```

Как ищет? ANN (Approximate Nearest Neighbors)

Точный поиск по всем векторам → $O(n)$ — слишком долго

ANN-алгоритмы:

- HNSW (Hierarchical Navigable Small World)
 - граф с иерархическими уровнями
 - $O(\log n)$ поиск, ~99% точность

Сравнение подходов к поиску

Тип поиска	Точное совпадение	Полнотекстовый	Семантический	SQL + Vector
'Оплата' ≈ 'payment'	Нет ✗	Нет ✗	Да ✓	Да ✓
Фильтры по полям	Да ✓	Частично	Payload filter	Полный SQL ✓
JOIN между таблицами	Да ✓	Нет ✗	Нет ✗	Да ✓
Скорость	~1-5ms	~5-50ms	~50-200ms	~5ms + ~100ms

HNSW: Hierarchical Navigable Small World

Идея: многоуровневый граф, где верхние уровни — «хайвеи», нижний — все данные

Уровень 2 (fast highway) — несколько узлов, длинные рёбра → быстрый переход в нужный регион

Уровень 1 (medium) — $\sqrt{N}$ узлов, средние рёбра

Уровень 0 (все данные) — N точек, короткие рёбра → точный локальный поиск

Алгоритм поиска:

1. Стартуем на верхнем уровне → жадно идём к ближайшему соседу
2. Спускаемся на уровень ниже → продолжаем в более плотном графе
3. На уровне 0 — финальный локальный поиск в окрестности

Qdrant — нативные фильтры в HNSW

Фильтрация прямо в графе: обходим только `source_type='code'`.

Не post-hoc! Точность не теряется.

Сложность:

Построение: $O(N \log N)$

Поиск: $O(\log N)$

Почему мы выбрали Qdrant

- ▶ Написан на Rust — минимальный memory footprint, без GC-пауз
- ▶ Self-hosted — данные не покидают наш контур (Jira, код, docs)
- ▶ Payload filtering во время поиска, не после — нативная фильтрация при обходе HNSW-графа
- ▶ Docker-запуск одной командой: `docker run qdrant/qdrant`
- ▶ Бесплатный open-source, Cloud-версия для масштабирования
- ▶ REST + gRPC API, SDK для Python, JS, Go, Rust

Qdrant = баланс: бесплатный self-hosted + производительность Rust + нативные payload-фильтры

Сравнение: Qdrant vs Pinecone vs Chroma vs pgvector

	Qdrant	Pinecone	Chroma	pgvector
Язык	Rust	Managed	Python	C
Деплой	Self / Cloud	Cloud only	Embedded	PG extension
Производительность	Очень высокая	Высокая	Средняя	Средняя
Фильтры	Нативные в HNSW	Да	Post-hoc	SQL WHERE
Настройка	Docker 1 cmd	Signup	pip install	CREATE EXT
Стоимость	Бесплатно	\$\$\$\$	Бесплатно	Бесплатно
Лучше для	Продакшн	No-ops	Прототип	Уже на PG

* Post-hoc (Chroma): сначала ищем Top-K векторов, затем применяем фильтры. При малых выборках теряем точность — нужные документы могут не попасть в Top-K.

** No-ops (Pinecone): полностью managed cloud-сервис, нулевые операционные затраты. Минус: данные уходят в облако, нет self-hosted варианта.

Теория: Cosine Similarity — как измеряем похожесть

Формула

$$\cos(A, B) = (A \cdot B) / (||A|| \times ||B||) = \Sigma(A_i \times B_i) / (\sqrt{\Sigma(A_i^2)} \times \sqrt{\Sigma(B_i^2)})$$

1.0

**Идентичные
документы**

Один и тот же текст

0.7-0.9

Высокая похожесть

Тот же смысл,
другие слова

0.3-0.7

**Умеренная
похожесть**

Связанные темы

0.0-0.3

Низкая похожесть

Разные темы

-1.0

Противоположные

Антонимы /
контраст

На практике в нашей RAG-системе:

- ▶ Порог релевантности: score > 0.3 — ниже отсекаем как нерелевантное
- ▶ Топ-K = 10-30 результатов
- ▶ Результаты ре-ранжируются по source_weights (git > code > jira > confluence)

Ограничения классического RAG

Проблемы

- ▶ Нет структурированных фильтров:
статус, проект, assignee, дата
- ▶ Confluence может быть устаревшей —
а vector score будет высоким
- ▶ Один размер не подходит всем запросам:
архитектор ≠ тестировщик ≠ аналитик
- ▶ Нет обратной связи:
система не учится на ошибках
- ▶ Chunk size trade-off:
маленькие = потеря контекста,
большие = шум в результатах

Наши решения

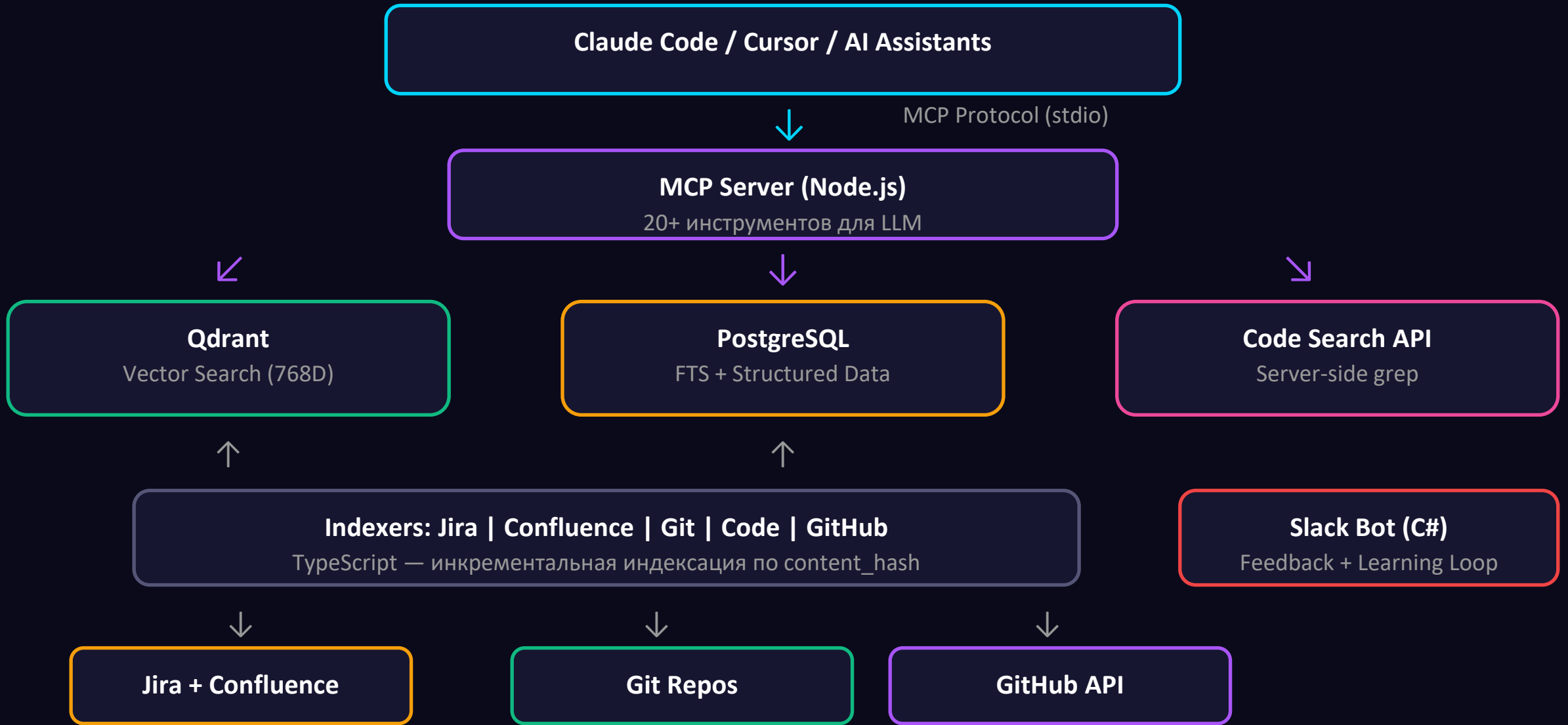
- ▶ Гибридный поиск: SQL + Vectors
PostgreSQL FTS для точных + Qdrant для смысла
- ▶ Структурированные фильтры:
WHERE status = 'Done' AND project = 'PROJ'
- ▶ Source Weights:
Confluence × 0.8, Git Commits × 1.4
- ▶ Ролевые агенты:
search_as_architect, search_as_qa, ...
- ▶ Feedback loop через Slack:
оценка → коррекция → сохранение в RAG
- ▶ Оптимальный chunk:
1500 символов + 200 overlap (эмпирически)

Часть 2

Гибридный поиск

PostgreSQL FTS + Qdrant Vectors

Архитектура системы



Конвейер индексации (Data Ingestion Pipeline)



Результат:

99% skip rate

при инкрементальной индексации

10K+ Jira issues

за < 5 минут (incremental)

Оркестрация + 5 индексеров

Оркестрация (index-all.ts)

1. Qdrant health check
2. Загрузка реестра индексеров
3. Последовательный запуск:
Jira → Confluence → Git → Code → GitHub
4. Post-indexation (термины)

5 индексеров — каждый со своей стратегией

Jira

REST API v3, JQL, 50/page

Confluence

REST API, 25/page

Git Local

git log --all (локально)

GitHub

REST API, 100/page

Code

fs.readdir рекурсивно

Embedding Provider + Dual Write

Embedding Provider (фабрика)

```
// shared/src/embeddings/factory.ts
switch (EMBEDDING_PROVIDER) {
  case "ollama":
    // nomic-embed-text, 768D
    // Бесплатно, Auto-pull модели
  case "openai":
    // text-embedding-3-small, 1536D
    // Батч до 100, ~$0.02/1M tok
}
```

Dual Write: запись в 2 хранилища

```
// jira-full-indexer.ts
const needsQdrant = await isDocumentChanged(
  type, id, hash);
const needsPg = await shouldUpdateJiraIssue(
  key, hash);
if (needsQdrant)
  await upsertDocument({vector, payload});
if (needsPg)
  await upsertJiraIssue(pgInput);
```

4 подхода к чанкингу

Fixed Size

Режем каждые N символов

+ Просто, предсказуемо

- Рвёт предложения и функции

Paragraph-based

Разбиваем по абзацам (`\n\n`)

+ Сохраняет мысль автора

- Только для документации

Recursive Splitting

LangChain: абзацы → предложения → слова

+ Универсальный

- Не учитывает структуру кода

Semantic Boundary

По границам функций / классов

+ Чанк = логическая единица

- Нужны regex под каждый язык

Наша реализация + стратегия по источникам

Реализация (indexer/src/chunker.ts)

```
// Semantic Boundary + Size Limit + Overlap
for (let i = 0; i < lines.length; i++) {
  const isBoundary = isSemanticBoundary(line);
  if (currentSize > maxChunkSize) saveChunk();
  if (isBoundary && currentSize > max/2)
    saveChunk(); // Граница
  // Overlap: последние 200 chars
}
```

Семантические границы:

C#: namespace, class, interface, struct, enum, record

TS: function, class, interface, export, type, const

Стратегия по источникам

Код:

Semantic Boundary, 1500 chars

200 overlap, min 100 chars

Confluence:

Paragraph-based, 2000 chars

Заголовок в каждом чанке

Jira / Git:

Без чанкинга — целиком

Достаточно короткие

6 типов source_type — каждый со своим payload

jira

project, issue_type, status, components, labels, updated

git_commit

repo, commit_hash, author, email, date, subject, jira_keys

code

repo, file_path, language, chunk_index, start_line, end_line

confluence

space_key, page_title, version, chunk_index, updated

decision

title, context, decision, reasoning, jira_keys (от LLM)

domain_term

term, definition, aliases (от LLM через add_domain_term)

Фильтрация + запись знаний

Нативная фильтрация в HNSW-графе

```
// search_as_architect → Qdrant filter:
qdrant.search(collection, {
  vector: embedding,
  filter: {
    must: [{
      key: "source_type",
      match: {any: ["code", "git_commit"]}
    }]
  },
  score_threshold: 0.3
});
```

Путь записи: знания в Qdrant

```
// add_decision (MCP tool)
const text = `${title}. ${context}`;
const embedding = await getEmbedding(text);
await qdrant.upsert(collection, {
  points: [{
    id: uuidv4(),
    vector: embedding,
    payload: {
      source_type: "decision",
      title, context, decision
    }
  }]
});
```

Весовые коэффициенты источников

Не все источники одинаково надёжны. Git > Code > Jira > Confluence.

git_commit		× 1.4	Факт из кодовой базы
code		× 1.3	Текущее состояние кода
decision		× 1.1	Арх. решения (ручные)
jira		× 1.0	Базовый вес
domain_term		× 1.0	Терминология
confluence		× 0.8	Может быть устаревшей!

```
// mcp-server/src/index.ts – применение весов при поиске
results.forEach(result => {
  result.weighted_score = result.vector_score * SOURCE_WEIGHTS[result.source_type];
});
```

Qdrant Graph View — наши данные в векторном пространстве

ВСТАВЬ СКРИНШОТ
Qdrant Dashboard → Graph View

- **Jira-задачи**
Кластеры по проектам — семантически близкие задачи группируются
- **Git-коммиты**
Коммиты одной фичи собираются вместе в пространстве
- **Confluence**
Документация — отдельные кластеры по доменам
- **Decisions**
Архитектурные решения — ручные записи экспертов

Payload filtering: при поиске `source_type = 'code'` — Qdrant обходит только зелёные точки в HNSW-графе

PostgreSQL: схема данных + Full-Text Search

```
CREATE TABLE jira_issues (  
  id SERIAL PRIMARY KEY,  
  issue_key VARCHAR(50) UNIQUE NOT NULL,  
  project_id INT REFERENCES jira_projects(id),  
  summary TEXT NOT NULL,  
  description TEXT,  
  status VARCHAR(100),  
  status_category VARCHAR(50),  
  assignee_id INT REFERENCES persons(id),  
  story_points DECIMAL(5,2),  
  content_hash VARCHAR(64), -- SHA256  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
);  
);
```

Связи между таблицами

```
persons ← jira_issues (assignee_id)  
        |  
        | jira_issue_links (linked_issue_key)  
        | jira_issue_labels (label)  
        | jira_issue_components  
        |  
        | git_commits ← git_commit_jira_refs  
        |  
        | confluence_pages ←  
confluence_page_labels
```

PostgreSQL FTS: GIN-индекс, tsvector, ts_rank

GIN-индекс (Generalized Inverted Index)

Как предметный указатель книги: слово $\rightarrow$ {строка₁, строка₂, ...}.

Поиск по слову: $O(1)$ lookip вместо $O(N)$ full scan по всем строкам.

tsvector — токенизация + стемминг

'Оплата заказа №123' $\rightarrow$ 'заказ':2 'оплат':1

Удалены стоп-слова (предлоги), корни нормализованы (оплатить $\rightarrow$ оплат).

tsquery: to_tsquery('оплат:* & заказ:*') — ищем по корням, не по точному слову.

ts_rank — ранжирование по релевантности

TF-подобный scoring: частота вхождения $\times$ позиция в документе $\rightarrow$ float [0..1].

Чем раньше слово встречается и чем чаще — тем выше rank.

~1–10ms на 10K+ записей

Только GIN lookip по индексу. Без full scan. Работает без настройки «из коробки» в PG.

Гибридный поиск в действии

Запрос: "Как мы валидируем платежи?"

PostgreSQL Path → Tier 1 (приоритет)

```
SELECT issue_key, summary,  
       ts_rank(  
         to_tsvector('english', summary),  
         plainto_tsquery('payment validation')  
       ) AS rank  
FROM jira_issues  
WHERE to_tsvector('english', summary)  
      @@ plainto_tsquery('payment validation')  
ORDER BY rank DESC  
LIMIT 20;
```

- ▶ Скорость: ~5ms
- ▶ Точные совпадения слов
- ▶ НЕ найдёт "оплата" по "payment"

Qdrant Path → Tier 2 (семантика)

```
const embedding = await getEmbedding(  
  "Как мы валидируем платежи?"  
);  
  
const results = await qdrant.search({  
  vector: embedding,  
  limit: 30,  
  filter: {  
    must: [{ key: "source_type",  
              match: { any: ["code", "git_commit"] }  
            }]  
  }  
});
```

- ▶ Понимает семантику
- ▶ "оплата" ≈ "payment" ✓
- ▶ Медленнее: ~100ms

Часть 3

MCP Protocol

знания как инструменты для LLM

3 паттерна вызова Claude в нашей системе

1. Claude CLI из C#

Slack Bot (ASP.NET)

```
// ClaudeService.cs
var startInfo = new
ProcessStartInfo
{
    FileName = "claude",
    Arguments = string.Join(" ",
        "--print",
        "--dangerously-skip-
permissions",
        "--model", "sonnet",
        "--output-format",
        "stream-json"
    )
};
```

2. Claude CLI из Python

Telegram Bot (python-telegram-bot)

```
# telegram_bot.py
cmd = [
    "claude", "-p", prompt,
    "--model", "sonnet",
    "--allowedTools",
    "search_patterns",
    "search_by_person",
    "--disallowed-tools",
    "Bash","Edit","Write","Read"
]
subprocess.run(cmd,
timeout=120)
```

3. Anthropic SDK

Агенты + Аватар-сервер

```
// TypeScript / Python SDK
import Anthropic from
"@anthropic-ai/sdk";

const response = await
anthropic.messages.create({
    model: "claude-sonnet-4",
    max_tokens: 4096,
    system: systemPrompt,
    messages: [{ role: "user",
        content: query }]
});
```

Теория: MCP — Model Context Protocol

Открытый стандарт от Anthropic (2024) для подключения LLM к внешним инструментам и данным.
Как USB-C для AI — один стандартный «разъём» для любого LLM-клиента и любого инструмента.

Транспорт: stdio

Дочерний процесс LLM-клиента.
Никакого HTTP, портов, firewall.
Общение через stdin/stdout.

.NET аналог:
IHostedService + Channel<T>

Протокол: JSON-RPC 2.0

Два главных метода:

- tools/list → список инструментов
- tools/call → вызов инструмента

.NET аналог:
SignalR Hub / gRPC service

Безопасность

Нет HTTP = нет поверхности атаки
Данные не покидают машину
Не нужен auth/HTTPS/CORS

Критично для enterprise:
внутренний код не утекает

MCP vs REST API

Deploy
Latency
Security

MCP
npm install + .mcp.json
~0ms (stdio pipe)
Нет сети

REST API
Docker + K8s + Ingress
~5-50ms (network)
Auth + HTTPS + CORS

Регистрация MCP-инструментов

```
// mcp-server/src/index.ts
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";

const server = new McpServer({ name: "domain-rag", version: "1.0" });

// Каждый tool — это "endpoint" для LLM
server.tool(
  "search_as_architect",
  "Search code and git commits for technical patterns",
  {
    query: z.string().describe("Technical search query"),
    limit: z.number().optional().default(10),
    repo: z.string().optional()
  },
  async ({ query, limit, repo }) => {
    const results = await searchWithWeights({
      query,
      limit,
      sourceTypes: ['code', 'git_commit'],
      filters: repo ? { repo } : undefined
    });
    etc...
  }
);
```

Как LLM выбирает tool?

LLM видит JSON Schema каждого tool:

- name — уникальный идентификатор
- description — когда использовать
- inputSchema — параметры

Claude читает описание и решает:
«это вопрос про код → search_as_architect»

20+ инструментов в 4 уровнях приоритета

Tier 1 — PostgreSQL (приоритет)

- `search_structured` — единый FTS по всем таблицам
- `search_jira_structured` — Jira + фильтры (`status`, `project`, `assignee`)
- `get_jira_by_project` — бэклог проекта
- `get_commits_for_jira` — коммиты → задачи (через `jira_refs`)
- `search_persons` — люди + их активность
- `raw_sql` — SELECT запросы (только `read-only`!)
- `get_db_stats` — статистика базы

Tier 2 — Qdrant (ролевые агенты)

- `search_as_analyst` — бизнес-контекст (Jira + Confluence)
- `search_as_architect` — код + коммиты
- `search_as_qa` — тесты + баг-репорты
- `search_as_docs` — только Confluence
- `search_knowledge` — все источники с весами

Tier 3 — Knowledge Mgmt

- `add_domain_term` — добавить в глоссарий
- `add_decision` — записать арх. решение
- `search_decisions` / `search_terms` — поиск

Tier 4 — Code Search (дорогой!)

- `search_server_code` — `server-side grep`
- `list_server_repos` — доступные репозитории

PostgreSQL Handlers + Полевые агенты

1 PostgreSQL Handlers

~5ms | Tier 1

```
// search_jira_structured → pgSearchJira()
const results = await pgSearchJira(query, {
  projectKey, status, limit // SQL-фильтрация
});
// raw_sql → защита: только SELECT, блокируем insert/delete/drop
```

2 Полевые агенты (Qdrant)

~100ms | Tier 2

```
// Все роли → одна функция, разные фильтры:
analyst:   sourceTypes: ["jira","confluence"]
architect: sourceTypes: ["code","git_commit"]
docs:      sourceTypes: ["confluence"]
// QA – особый: расширяет запрос + post-filter
const testQuery = `${query} test spec bug`;
```

Knowledge Management + Code Search

3 Knowledge Management

Write → Qdrant

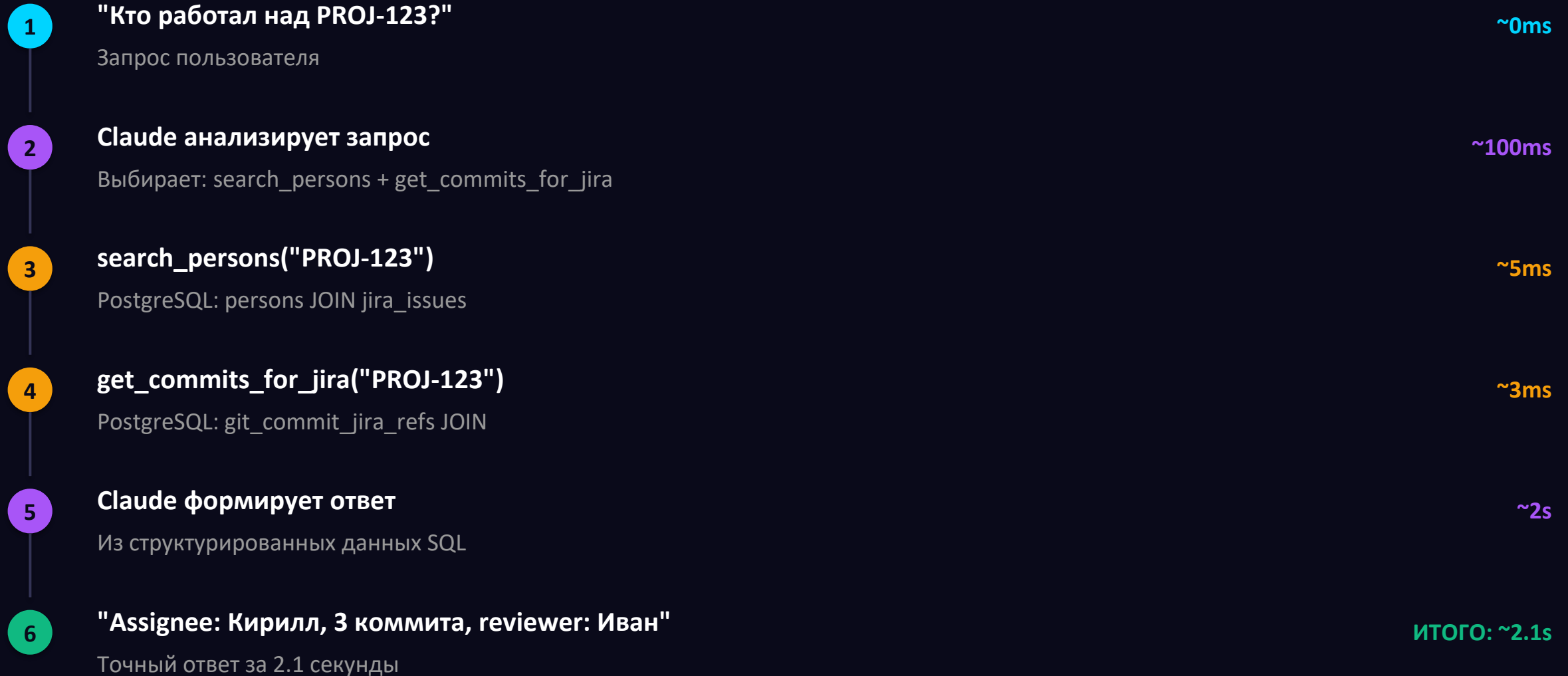
```
// add_decision – tools, которые ПИШУТ в Qdrant
const text = `${title}. ${context}. ${decision}`;
const embedding = await getEmbedding(text);
await qdrant.upsert(collection, { points: [{
  id: uuidv4(), vector: embedding,
  payload: { source_type: "decision", title, context }
}]});
```

4 Code Search API

HTTP | Tier 4

```
const response = await fetch(
  `${CODE_SEARCH_URL}/api/search`,
  { method: "POST", body: JSON.stringify({
    query, repos, extensions, limit
  }) }
);
```

Живой пример: запрос от начала до конца



Вручную: Jira → Git log → PR → ~5 минут | С RAG-агентом: ~2 секунды

Часть 4

Автономный агент

от поиска к действию

Как LLM вызывает инструменты?

1. LLM получает список tools + их JSON Schema
2. Пользователь задаёт вопрос
3. LLM решает: «мне нужны данные из tool X»
→ генерирует JSON-вызов tool
4. Клиент выполняет tool, возвращает результат
5. LLM формирует финальный ответ с контекстом

Agent = Tool Use в цикле: LLM сам решает, сколько итераций и какие tools вызвать.

Эволюция: RAG → MCP Tools → Автономный агент

RAG

Поиск контекста

LLM получает
релевантные документы



MCP Tools

Действия

LLM вызывает
инструменты



Agent

Автономность

LLM сам решает
что и когда вызвать

Мульти-агентная оркестрация

Orchestrator: `evaluate_agents()` → confidence score

Analyst

Бизнес-контекст
Jira + Confluence

conf: 0.3

Architect

Код + паттерны
code + git_commit

conf: 0.92 ✓

QA

Тесты + edge cases
test files + bugs

conf: 0.5

Docs

Документация
Confluence only

conf: 0.2

Как считается confidence?

$\text{score} = \text{cosine_sim}(\text{query_embedding}, \text{agent_domain_docs}) \times \text{source_weight}$

- Architect: query про код → высокое сходство с code/git_commit → 0.92
- Analyst: тот же query → низкое сходство с jira/confluence → 0.30
- Победитель: агент с max confidence. Если score < 0.5 — параллельный вызов всех агентов.

Feedback Loop: самообучение через Slack

1. Вопрос в Slack

"Как масштабировать X?"

2. Agent ищет в RAG

Claude + MCP → ответ

3. Оценка: 3/10

Плохой ответ! (< 5)

4. → Learning Channel

Вопрос уходит экспертам

5. Эксперт корректирует

Даёт правильный ответ

6. Save to RAG!

Decision + Domain Term

Система учится БЕЗ fine-tuning. Просто сохраняем правильные ответы как decisions в RAG.

C# код: LearningService

```
// slack-bot/src/TenenAgent/Services/LearningService.cs

public async Task HandleRating(string threadId, int score)
{
    if (score >= 5) {
        session.State = SessionState.Complete; // OK
        return;
    }
    // Плохой → пересылаем экспертам
    session.State = SessionState.AwaitingLearning;
    await ForwardToLearningChannel(session);
}

public async Task HandleExpertCorrection(string answer)
{
    // Сохраняем в RAG → следующий раз лучше!
    await _ragSave.SaveDecision(question, answer);
}
```

Цифровой двойник инженера

Агент, который понимает контекст как опытный инженер, действует самостоятельно и учится на обратной связи. Не замена — усиление.

Мониторинг

- ▶ Получает алерт из Grafana
- ▶ Ищет в RAG: «что было раньше?»
- ▶ Находит runbook в Confluence
- ▶ Определяет on-call инженера

Действие

- ▶ Создаёт Jira-тикет
- ▶ Уведомляет в Slack
- ▶ Предлагает fix из истории
- ▶ Записывает decision

Обучение

- ▶ Человек корректирует
- ▶ Сохраняет исправление
- ▶ Следующий раз —
- ▶ правильный ответ сразу

Сценарий: алерт "Database CPU spike"

1. Алерт → Grafana API → Slack thread
2. Agent спрашивает RAG: "что вызывало CPU spikes раньше?" → находит 3 инцидента
3. Находит runbook: "Проверить slow queries → pg_stat_activity"
4. Создаёт Jira: "DB-OPT: исследовать CPU spike" + assignee = on-call
5. Уведомляет on-call в Slack с контекстом прошлых инцидентов
6. Человек фиксирует → агент сохраняет: "CPU spike → missing index on orders.created_at"

Полный технологический стек

MCP Server	TypeScript + @modelcontextprotocol/sdk	LLM ↔ Tools интеграция
Vector DB	Qdrant (local / cloud)	Семантический поиск
Relational DB	PostgreSQL + FTS + pgvector	Структурированные запросы
Embeddings	Ollama (local) / OpenAI API	Векторизация текста
Indexers	TypeScript (Node.js)	Синхронизация Jira/Confluence/Git
Slack Bot	C# (ASP.NET Core)	Feedback + Learning Loop
Digital Twin	Python + ElevenLabs + Qdrant	Голосовой аватар
Deploy	Docker Compose	Оркестрация контейнеров

Как начать (для .NET разработчика)

1

Шаг 1: Попробуй

Установи Claude Code / Cursor. Подключи любой MCP-сервер. Просто попробуй задать вопрос — и увидишь, как LLM вызывает tools.

2

Шаг 2: Индексируй

Начни с одного источника (Jira или Git). Ollama + Qdrant локально.
docker-compose up → npm run index → profit.

3

Шаг 3: Добавь структуру

PostgreSQL для фильтров и JOIN-ов. Гибридный поиск = магия.
FTS бесплатный — уже встроен в PostgreSQL.

Как начать (для .NET разработчика) — шаги 4–

5

Шаг 4: Feedback loop

Slack/Teams бот для оценок. Learning Service на C# — ваш стек.

Без fine-tuning, без prompt engineering — просто сохраняй правильные ответы.

Шаг 5: Автономность

Агент, который мониторит, ревьюит и учится. Цифровой двойник.

Это не конец — это начало.

Весь стек — open source, self-hosted, бесплатный.

Docker + TypeScript + PostgreSQL + Qdrant + Claude API → цифровой двойник за 2 недели.

Ключевые выводы

1

Классический RAG — только начало. Гибридный поиск (SQL + Vectors) даёт и точность, и понимание семантики.

2

MCP превращает знания в инструменты. LLM сам выбирает нужный tool по описанию — не нужна ручная маршрутизация.

3

Feedback loop — единственный способ не деградировать. Система должна учиться на своих ошибках без fine-tuning.

4

Content Hash = 99% экономии ресурсов.
Не пересчитывайте embeddings, если контент не изменился.

5

Цифровой двойник = RAG + личность + голос.
Личность — это данные, не промпт-инженерия.

Плюсы подхода

- ✓ **Self-hosted** Qdrant, PostgreSQL, Ollama — данные не покидают контур
- ✓ **99% skip rate** Content hash экономит сотни долларов на embeddings
- ✓ **Гибридный поиск** SQL + вектора — LLM сам выбирает инструмент
- ✓ **Обучение без fine-tuning** Feedback loop: decisions сохраняются в RAG
- ✓ **MCP стандарт** Один сервер → Claude Code, Cursor, Slack, Telegram

Путь 1: Мультимодальный RAG

Краткосрочно (3 мес)

Проблема: диаграммы, скриншоты UI, ER-схемы в Confluence и Jira — невидимы для RAG.

Решение: Vision-пайплайн в индексере

```
// confluence-indexer.ts — новый шаг
if (isImage(attachment)) {
  const description = await claude.messages.create({
    model: "claude-sonnet-4",
    messages: [{
      role: "user",
      content: [{ type: "image", source: attachment },
        { type: "text", text: "Опиши эту схему" }]
    }]
  });
  await upsertDocument({ source_type: "image_description",
    content: description, ... });
}
```

Путь 2: Проактивный автономный агент

Стратегически

Grafana алерт

CPU базы на 95%
Webhook → агент

RAG поиск

«Что вызывало
высокий CPU раньше?»

Найден decision

«Помогла перезагрузка
connection pool»

Создаёт Jira

Тикет с описанием
и предлагаемым fix

Уведомляет on-call

Slack: полный контекст
+ предложенное решение

Human confirms

On-call подтверждает
→ агент применяет fix

Human-in-the-loop: агент ПРЕДЛАГАЕТ решение, человек ПОДТВЕРЖДАЕТ.

Бонус: инструменты, которые ускоряют разработку

Два открытых инструмента, которые я использую каждый день вместе с Claude Code

/graphify

Знания в граф

Любую папку с кодом, документами и статьями превратить в наведируемый граф знаний. Сам находит связи между концепциями, которые ты не заметил бы.

ruflo

Рой агентов

Платформа мультиагентной оркестрации поверх Claude Code: 100+ специализированных агентов, self-learning, общая память, автомаршрутизация задач.

/graphify — любой input в навилируемый граф знаний

Код, PDF-статьи, заметки, твиты, скриншоты — кидаешь в папку, команда /graphify строит граф

/raw

Папка с чем
удобно: код, PDF,
заметки, картинки

Extract

LLM тащит
сущности и связи
из каждого файла

Cluster

Community
detection —
группы концепций

HTML / JSON

Интерактивный
граф + GraphRAG-
ready JSON

Что умеет Claude только с graphify:

Persistent graph

Граф живёт в graphify-out/graph.json.
Спрашивай через недели без повторного
чтения.

Честный аудит

Каждая связь помечена EXTRACTED,
INFERRED или AMBIGUOUS. Знаешь где
факт, где догадка.

Cross-document surprise

Находит связи между файлами, о которых
ты никогда бы не догадался спросить
напрямую.

ruflo — рой агентов поверх Claude Code

Раньше был claude-flow. Ставим CLI, ruflo init — и у Claude Code появляется рой специализированных агентов.

100+ агентов

Кодер, тестер, ревьюер,
DevOps, security,
архитектор, документатор
— у каждого свой промпт

Swarm

Queen-координатор
декомпозирует задачу,
раздаёт агентам.
Hierarchical-mesh + Raft

Self-learning

Запоминает удачные
патерны, меняет
маршрутизацию. Память
на RuVector HNSW

Hooks

UserPromptSubmit,
PreToolUse, PostToolUse.
Роутинг по содержимому
— без ручного выбора

Что меняется в разработке:

Без ruflo

Одна задача — один Claude.
«Добавь документацию + тесты + рефакторинг»
— делает последовательно, теряет контекст.

С ruflo

Одна команда → Queen разбирает задачу →
арх-агент + test-агент + docs-агент работают параллельно,
делясь общей памятью. Время × 0.3, качество стабильнее.

`npm i -g ruflo` → `ruflo init --full` → открываешь Claude Code и пишешь задачу

Спасибо!

Вопросы? Мой цифровой двойник готов ответить

Кирилл Сухоруких

Senior Fullstack Developer

SoftLight

Telegram @Godbaron1337

GitHub github.com/KirillMaster

Email sukhorukikh1996@gmail.com